04 / LEARNING GUIDE

Pseudocode Guide

EX28 · From mathematical reasoning to algorithm design

> **Reading order:** 01 Engineering & Math Theory → 02 Python Reference → 03 Exercise → 04 Pseudocode → programming. This guide belongs to EX28, class 18.

1. Purpose

Pseudocode captures the logic of your program before choosing exact Python syntax. Follow the mathematical process: identify inputs, check dimensions, select operations, calculate outputs, and verify answers.

2. Five-step mental model

```
INPUTS -> DIMENSIONS -> OPERATIONS -> RESULTS -> VALIDATION
```

If the dimensions do not allow an operation, stop and revise the mathematical setup. A matrix of shape (2,3) cannot be multiplied by a length-2 vector as A @ x.

3. Questions to ask before coding

- What are the **inputs**: matrices, vectors, scalars?
- What is the **shape** of each input?
- Do I need addition, transpose, element-wise multiplication, or a matrix product?
- What should the **output shape** be? Are there physical units?
- Which answers can I calculate by hand for validation?

4. Guided example A0: matrix times vector

Given the example matrix A and vector x, we want a two-component output.

```
START
  Define matrix A
  Define vector x
  Inspect the shape of A and x
  Compute y by matrix-vector multiplication
  Display A, x, y and their shapes
  Compare y against the manual answer (0, 2)
END
```

This is pseudocode, not a full Python implementation. The central calculation will use @ later.

5. A1 template: independent practice

```
START
  INPUTS: C = [...], D = [...], x = [...]
  DIMENSIONS: inspect ...
  OPERATION 1: matrix addition ...
  OPERATION 2: transpose ...
  OPERATION 3: matrix times vector ...
  OPERATION 4: compare C times D, D times C ...
  OPERATION 5: multiply entries ...
  OUTPUTS: display ...
  VALIDATION: compare against ...
END
```

Fill in the missing decisions on paper or in comments before coding. Keep the reasoning that distinguishes element-wise multiplication from matrix multiplication.

6. Part B template: from physics to NumPy

Identify the inputs first: columns of A are actuator directions; components of u are force magnitudes in N. Then translate the physical linear combination into a matrix-vector product.

```
START
  Define unit directions e1 and e2
  Form A by placing e1 and e2 in its columns
  Define magnitudes u in N
  Calculate R using a matrix operation
  Calculate R again as the sum of two forces
  Compare both results
  Calculate the magnitude of R
  Print components, magnitude and units
END
```

7. Practical programming and validation tips

- **Choose the operation before the syntax:** first decide that you need matrix multiplication, then use @.
- **Use readable names:** for instance `resultant_force` or `transposed_matrix`.
- **Validate values and shapes:** the expected number of output components matters.
- **Avoid premature complexity:** one Python script is enough for EX28; you do not need modules yet.

> **Quick checklist:** Do I know the inputs? Can I write the algebra? Are shapes compatible? Can I predict the result?